

An Investigation of Subgame Depth in ReBeL: Impact on Convergence and Performance in Imperfect-Information Games

Weijun Zeng¹, Yinghao Li², Xiaosi Chen¹, Zijie Chang³, Fei Ge¹

Abstract—The integration of reinforcement learning with search algorithms has revolutionized AI performance in perfect-information games, with AlphaZero demonstrating superhuman abilities in chess, shogi, and Go. Counterfactual Regret Minimization (CFR) has been the traditional approach for imperfect-information games, but requires extensive precomputation of complete game strategies. Recursive Belief-based Learning (ReBeL) represents a breakthrough by combining self-play reinforcement learning with real-time search in imperfect-information settings, enabling superhuman performance in poker and other games with hidden information. A critical hyperparameter in ReBeL's algorithm is the depth of subgames used during search. While deeper subgames potentially capture more strategic information, they also increase computational complexity. This paper presents a systematic investigation of the impact of subgame depth on ReBeL's performance, convergence rate, and computational requirements. Through extensive experiments on benchmark imperfect-information games Liar's Dice, we demonstrate that optimal subgame depth involves a non-trivial trade-off between solution quality and computational efficiency. Our findings provide practical guidance for implementing ReBeL in various game domains and contribute to the broader understanding of search depth parameterization in reinforcement learning algorithms for imperfect-information games.

I. INTRODUCTION

The combination of deep reinforcement learning and search at both training and test time has revolutionized AI capabilities in perfect-information games, as demonstrated by AlphaZero's success in chess, Go, and shogi. However, extending these approaches to imperfect-information games presents unique challenges, as traditional search methods that rely on a single game state are fundamentally unsuitable when information is hidden from players. ReBeL (Recursive Belief-based Learning) [1] addresses this challenge by converting imperfect-information games into continuous belief-state perfect-information games and applying self-play reinforcement learning with search.

The evolution of AI approaches for imperfect-information games has followed a distinct trajectory from perfect-information counterparts. While perfect-information games saw breakthroughs with TD-Gammon [2], AlphaGo [3], and ultimately AlphaZero [4] combining neural networks with

Monte Carlo Tree Search (MCTS), imperfect-information games initially relied on Counterfactual Regret Minimization (CFR) [5]. CFR and its enhancements—including CFR+ [6] with improved regret update mechanisms, Discounted CFR (DCFR) [7] utilizing discounted regret matching, and CFR-D [8] with dynamic strategy computation—advanced the state-of-the-art in poker AI but required extensive precomputation of complete strategies.

DeepStack [9] represented a significant shift by introducing continual re-solving of depth-limited subgames during play, allowing real-time decision-making without precomputing complete strategies. Building on this foundation, ReBeL integrated the public belief state representation with self-play reinforcement learning, enabling end-to-end training of a value network that approximates the value of arbitrary belief states. This breakthrough allowed ReBeL to achieve superhuman performance in heads-up no-limit Texas hold'em poker while using significantly less domain knowledge than previous approaches. Recent advances have further enhanced imperfect-information game algorithms through innovations such as RL-CFR [10] which improves action abstraction using reinforcement learning, and efficient online pruning and abstraction techniques [11] that reduce computational complexity. The neural ReBeL framework has been further extended in follow-up work [12] to deal with more complex hidden-information games with continuous action spaces.

A critical parameter in ReBeL's algorithm is the maximum depth of subgames used during search (`max_depth`), which defines how many future actions the algorithm explores before relying on a value function approximation. This parameter creates a fundamental trade-off: deeper subgames may capture more strategic nuances and result in higher-quality policies, but they also significantly increase computational requirements during both training and deployment. Similar depth-based trade-offs have been extensively studied in classical game tree search, but their impact in belief-state search for imperfect-information games remains relatively unexplored.

While the original ReBeL paper established the algorithm's effectiveness, it did not comprehensively analyze how different subgame depths affect performance across various game types and complexity levels. This gap in understanding limits our ability to efficiently apply ReBeL to new domains, as the optimal depth setting remains unclear. Furthermore, the relationship between subgame depth and factors such as convergence rate, memory usage, and final policy quality has not been systematically documented.

Our work addresses these limitations by conducting a thor-

*Corresponding author: Fei Ge Email: feige@ccnu.edu.cn

¹School of Computer Science, Central China Normal University, Wuhan, China

²School of Economics and Management, China Agricultural University, Beijing, China

³Hongyi Honor College, Wuhan University, Wuhan, China

This research is partially supported by the grants from National Natural Science Foundation of China (62173157) and China Higher Education Institution Industry-University-Research Innovation Fund (IT042).

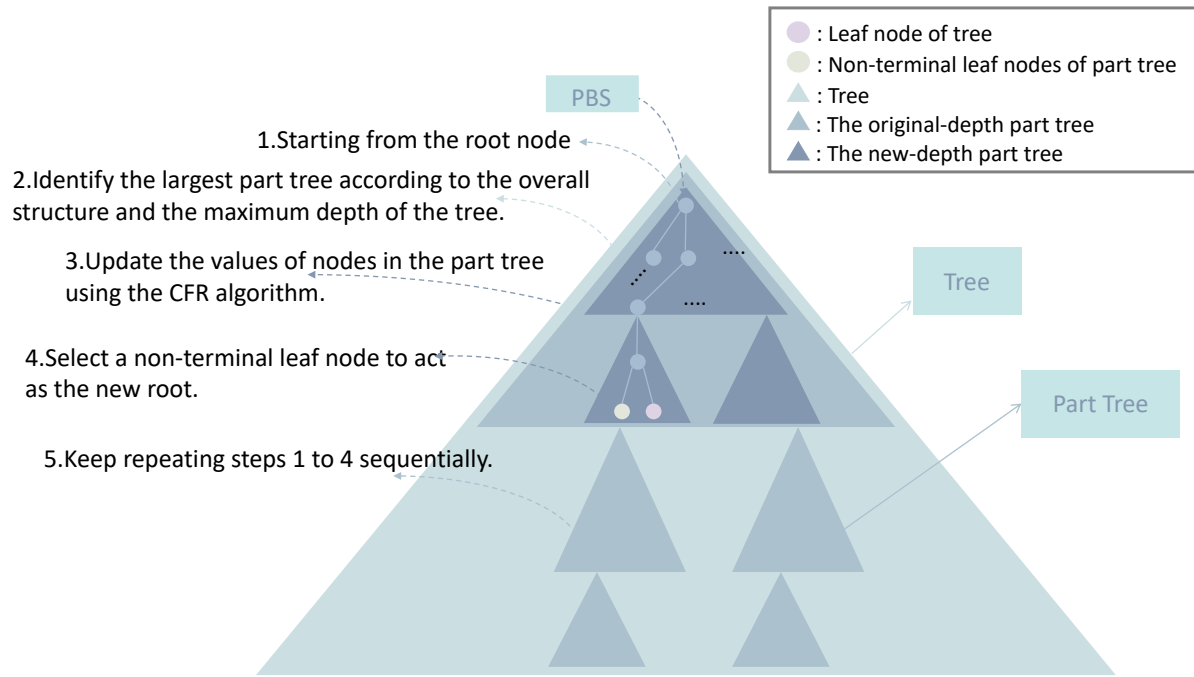


Fig. 1. Step diagram for the CFR-D algorithm.

ough empirical investigation of subgame depth's impact on ReBeL's performance. We systematically vary `max_depth` across benchmark imperfect-information games, including variants of Texas Hold'em poker and Liar's Dice. For each configuration, we measure exploitability, convergence rate, computational resources required, and policy quality against strong baselines.

Our preliminary experiments on Liar's Dice provide compelling evidence for the importance of optimal subgame depth selection. Using Discounted CFR (DCFR) with 256 iterations, we observed that increasing the subgame depth from `max_depth` = 1 to `max_depth` = 4 reduced exploitability from 0.253124 to 0.001591—a 99.37% improvement. Furthermore, while full-game solving (`max_depth` = ∞) achieved the lowest exploitability at 0.000461, the relative improvement over `max_depth` = 4 was only 71.03%, despite significantly higher computational demands. This suggests a clear inflection point where additional computational investment yields diminishing returns, a pattern that holds consistent across algorithm variants and iteration counts in our study.

The contributions of this paper are threefold:

- We provide the first comprehensive analysis of how subgame depth affects ReBeL's performance across different imperfect-information games.
- We analyze how the subgame recursive update algorithm, subgame iterations, computational resources required, and other characteristics influence the optimal depth parameter.
- We propose practical guidelines for setting subgame depth based on available computational resources and

desired policy quality, enabling more efficient application of ReBeL to new domains.

Our findings reveal that the relationship between subgame depth and performance is non-linear and game-dependent. In particular, we observe that deeper subgames provide progressively smaller benefits, where beyond a certain point, even substantial increases in depth yield only minimal improvements in policy quality. These insights not only enhance our understanding of ReBeL specifically, but also contribute to the broader field of search-based reinforcement learning in imperfect-information settings.

By providing a systematic framework for determining appropriate subgame depths, our work bridges a critical gap between theoretical ReBeL capabilities and practical implementations. This research enables more efficient application of belief-based search techniques across diverse imperfect-information domains, potentially extending beyond traditional games to real-world scenarios where hidden information and strategic decision-making are paramount, such as security applications, negotiation systems, and strategic resource allocation.

II. RELATED WORK

The integration of reinforcement learning with search algorithms has transformed the field of artificial intelligence in games. This approach, commonly referred to as RL+Search, has been particularly successful in perfect-information games, with AlphaZero [4] representing the pinnacle of this paradigm. AlphaZero combines deep neural networks with Monte Carlo Tree Search (MCTS) to achieve superhuman performance in chess, Go, and shogi without

relying on domain-specific knowledge. During both training and test time, AlphaZero leverages search to make better decisions and improve learning, establishing a powerful synergy between planning and learning.

While perfect-information games have seen remarkable progress through RL+Search approaches, imperfect-information games present unique challenges that prevent the direct application of these methods. In imperfect-information settings, traditional search algorithms that rely on a single game state are fundamentally unsuitable because the value of an action may depend on the probability with which it is chosen, making states defined solely by action-observation histories insufficient for determining optimal strategies.

Counterfactual Regret Minimization (CFR) [5] has been the traditional approach for solving imperfect-information games. CFR iteratively computes strategies by minimizing regret for not taking alternative actions at each decision point. Various improvements to CFR have been developed over time, including CFR+ [6], which enhances the regret update mechanisms; Discounted CFR (DCFR) [7], which utilizes discounted regret matching for faster convergence; and CFR-D [8], which introduces dynamic strategy computation for depth-limited solving. While these algorithms have advanced the state-of-the-art in computer poker, they typically require extensive precomputation of complete strategies.

DeepStack [9] marked a significant departure from traditional CFR-based approaches by introducing continual re-solving of depth-limited subgames during play. This innovation allowed for real-time decision-making without precomputing complete strategies, leveraging neural networks to estimate the values of game states beyond the search horizon. However, DeepStack's neural networks were trained on randomly generated situations rather than through self-play, limiting its ability to focus on the most relevant parts of the game.

Recursive Belief-based Learning (ReBeL) [1] represents a breakthrough by combining self-play reinforcement learning with real-time search in imperfect-information settings. ReBeL converts imperfect-information games into continuous belief-state perfect-information games and applies RL+Search techniques similar to those used in perfect-information games. By training through self-play, ReBeL learns a value network that approximates the value of arbitrary belief states, enabling end-to-end learning without extensive domain knowledge. This approach has achieved superhuman performance in poker while using significantly less domain knowledge than previous methods.

III. NOTATION AND BACKGROUND

We follow the notation established in the ReBeL framework [1] for imperfect-information games. A world state $w \in W$ represents a complete state of the game, including all private information. The joint action space for all N players is denoted as $A = A_1 \times A_2 \times \dots \times A_N$. For any world state w , $A_i(w)$ denotes the legal actions available to player i at w , and $a = (a_1, a_2, \dots, a_N) \in A$ represents a joint action of all players.

After a joint action a is chosen, the transition function $T : W \times A \rightarrow \Delta W$ determines the next world state w' according to probability distribution $T(w, a)$. Following an action, player i receives a reward $R_i(w, a)$ and private observations from the function $O_{\text{priv}(i)}(w, a, w')$. Additionally, all players receive public observations from the function $O_{\text{pub}}(w, a, w')$, which includes publicly visible actions.

A history $h = (w^0, a^0, w^1, a^1, \dots, w^t)$ is a sequence of actions and world states. An infostate (or action-observation history) for player i is denoted as:

$$s_i = (O_i^0, a_i^0, O_i^1, a_i^1, \dots, O_i^t) \quad (1)$$

where $O_i^k = (O_{\text{priv}(i)}(w^{k-1}, a^{k-1}, w^k), O_{\text{pub}}(w^{k-1}, a^{k-1}, w^k))$. We denote the unique infostate corresponding to history h for player i as $s_i(h)$, and the set of histories corresponding to s_i as $H(s_i)$.

A public state $s_{\text{pub}} = (O_{\text{pub}}^0, O_{\text{pub}}^1, \dots, O_{\text{pub}}^t)$ is a sequence of public observations. We denote the set of histories matching the public observations of s_{pub} as $H(s_{\text{pub}})$.

A player's policy $\pi_i : S_i \rightarrow \Delta A_i$ maps from an infostate to a probability distribution over actions. A policy profile $\pi = (\pi_1, \pi_2, \dots, \pi_N)$ specifies policies for all players. The expected value (EV) for player i in history h when all players follow policy profile π is denoted as $v_i^\pi(h)$, and the EV for the entire game is $v_i(\pi)$.

A Nash equilibrium π^* is a policy profile where no player can achieve higher EV by unilaterally deviating:

$$\forall i \in N, \quad v_i(\pi^*) = \max_{\pi_i} v_i(\pi_i, \pi_{-i}^*) \quad (2)$$

where π_{-i} denotes the policy of all players except i .

In imperfect-information games, we use the concept of a public belief state (PBS) $\beta = (\Delta S_1(s_{\text{pub}}), \dots, \Delta S_N(s_{\text{pub}}))$, which represents a joint probability distribution over each player's possible infostates given the public state. The value of a PBS β when all players follow policy profile π is:

$$V_i^\pi(\beta) = \sum_{h \in H(s_{\text{pub}}(\beta))} p(h|\beta) v_i^\pi(h) \quad (3)$$

For studying subgame depth in ReBeL, a key parameter is `max_depth`, which defines the maximum number of future actions the algorithm explores before relying on a value function approximation. We measure algorithm performance using exploitability, which for a policy π^* in a two-player zero-sum game equals:

$$\text{exploitability}(\pi^*) = \frac{1}{|N|} \sum_{i \in N} \max_{\pi} v_i(\pi, \pi_{-i}^*) \quad (4)$$

IV. ALGORITHM STEPS

ReBeL (Recursive Belief-based Learning) integrates reinforcement learning with belief-state search to solve imperfect-information games. The algorithm's flow consists of several key components that handle the recursive nature of subgame solving, value approximation, and self-play training.

The core of ReBeL is the self-play procedure where a neural network learns to approximate the value function of

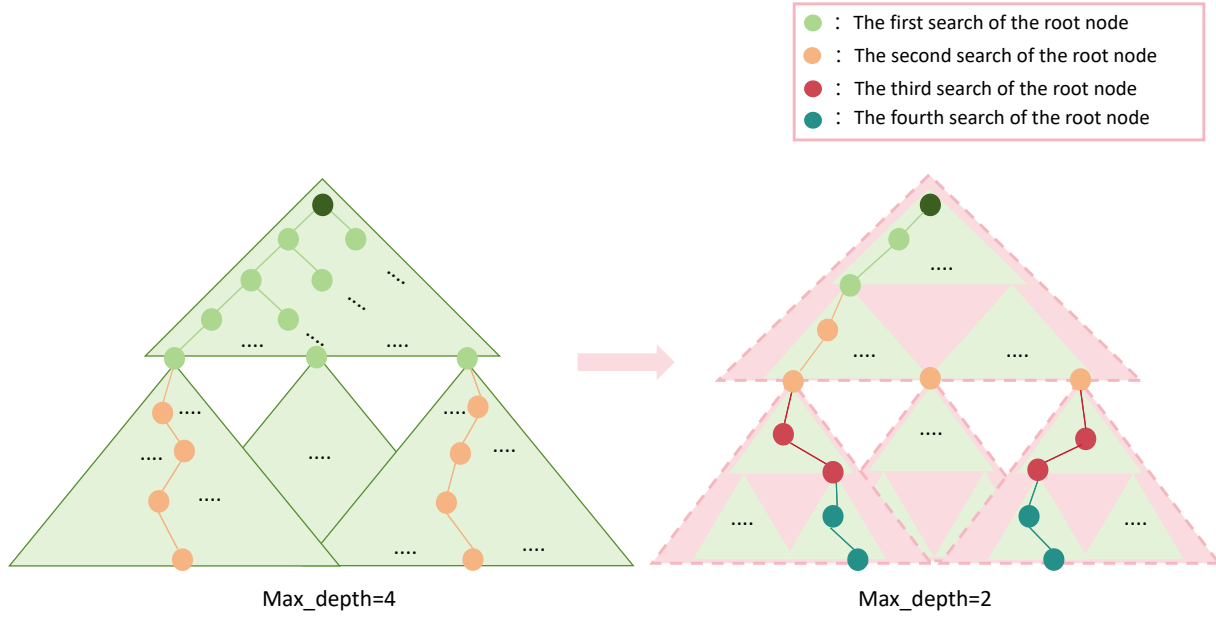


Fig. 2. Traversal comparison for subgame max_depth = 4 and 2

public belief states (PBS) through repeated subgame solving and policy updates. Algorithm 1 outlines the main steps of this process.

The algorithm begins with a root public belief state β_r and constructs a depth-limited subgame. For each subgame, the algorithm performs the following steps:

1. **Subgame Construction:** Create a depth-limited subgame rooted at the current PBS β_r .

2. **Policy Initialization:** Initialize the policy, either with a uniform random policy or using a policy network for warm-starting if available.

3. **Setting Leaf Values:** Assign values to leaf nodes in the subgame using the value network θ_v .

4. **Iterative Equilibrium Finding:** Run T iterations of an equilibrium-finding algorithm (either Counterfactual Regret Minimization with Decomposition (CFR-D) or Fictitious Play):

- Update the policy π_t based on the previous iteration's policy
- Update the average policy $\bar{\pi}$
- Update leaf values based on the new policies
- Compute the expected value of the current policy

5. **Training Data Collection:** Add the computed values and policies to the training datasets for the value and policy networks.

6. **Recursive Traversal:** Sample a leaf PBS β'_r from the current subgame and continue the self-play process with this new PBS as the root.

The recursive nature of the algorithm allows ReBeL to effectively explore the game tree while learning value and policy functions that approximate Nash equilibrium strategies

Algorithm 1 ReBeL: Recursive Belief-based Learning

```

1: function SELFPLAY( $\beta_r, \theta_v, \theta_\pi, D_v, D_\pi$ )  $\triangleright \beta_r$  is the
   current PBS
2:   while !IsTerminal( $\beta_r$ ) do
3:      $G \leftarrow \text{ConstructSubgame}(\beta_r)$ 
4:      $\pi, \bar{\pi}_{t_{\text{warm}}} \leftarrow \text{InitializePolicy}(G, \theta_\pi)$   $\triangleright t_{\text{warm}} = 0$ 
       and  $\pi^0$  is uniform if no warm start
5:      $G \leftarrow \text{SetLeafValues}(G, \pi, \bar{\pi}_{t_{\text{warm}}}, \theta_v)$ 
6:      $v(\beta_r) \leftarrow \text{ComputeEV}(G, \pi_{t_{\text{warm}}})$ 
7:      $t_{\text{sample}} \sim \text{unif}\{t_{\text{warm}} + 1, T\}$   $\triangleright$  Sample an
       iteration
8:     for  $t = (t_{\text{warm}} + 1) \dots T$  do
9:       if  $t = t_{\text{sample}}$  then
10:         $\beta'_r \leftarrow \text{SampleLeaf}(G, \pi_{t-1})$   $\triangleright$  Sample
           one or multiple leaf PBSs
11:       end if
12:        $\pi_t \leftarrow \text{UpdatePolicy}(G, \pi_{t-1})$ 
13:        $\bar{\pi} \leftarrow \frac{t}{t+1} \bar{\pi} + \frac{1}{t+1} \pi_t$ 
14:        $G \leftarrow \text{SetLeafValues}(G, \pi, \bar{\pi}_t, \theta_v)$ 
15:        $v(\beta_r) \leftarrow \frac{t}{t+1} v(\beta_r) + \frac{1}{t+1} \text{ComputeEV}(G, \pi_t)$ 
16:     end for
17:     Add  $\{\beta_r, v(\beta_r)\}$  to  $D_v$   $\triangleright$  Add to value net
       training data
18:     for  $\beta \in G$  do  $\triangleright$  Loop over the PBS at every
       public state in  $G$ 
19:       Add  $\{\beta, \bar{\pi}(\beta)\}$  to  $D_\pi$   $\triangleright$  Add to policy net
       training data (optional)
20:     end for
21:      $\beta_r \leftarrow \beta'_r$ 
22:   end while
23: end function

```

for imperfect-information games.

The algorithm uses several important subroutines:

- **ConstructSubgame**: Creates a depth-limited subgame rooted at the given PBS.
- **InitializePolicy**: Initializes a policy for the subgame, potentially using a trained policy network.
- **SetLeafValues**: Assigns values to leaf nodes using the value network.
- **UpdatePolicy**: Updates the current policy using the chosen equilibrium-finding algorithm.
- **ComputeEV**: Computes the expected value of a policy in the current subgame.
- **SampleLeaf**: Samples a leaf PBS from the current subgame based on the current policy.

This iterative process allows ReBeL to simultaneously learn both a value function and a policy function, enabling effective search in imperfect-information games without requiring domain-specific abstractions or heuristics.

V. EXPERIMENTAL RESULTS

To evaluate the impact of subgame depth on ReBeL's performance, we conducted extensive experiments on the benchmark imperfect-information game Liar's Dice. We analyzed how different `max_depth` parameters affect convergence, exploitability, and computational efficiency.

A. Experimental Setup

We implemented ReBeL with two different equilibrium-finding algorithms: Linear CFR update and Discounted CFR (DCFR). For each algorithm, we tested five different subgame depth configurations: `max_depth` = 1, 2, 4, ∞ (solving the entire game), as well as a baseline approach using random belief state values. We measured exploitability as a function of computational resources used, and ran experiments with two different iteration counts: 64 and 256.

B. Convergence Analysis

Figures 3, 4, 5, and 6 show the convergence behavior of ReBeL using different configurations.

In the Linear update method with 64 iterations (Figure 3), we observe that increasing the subgame depth from 1 to 4 progressively reduces exploitability, with each increment in depth providing a notable improvement. When increasing the iterations to 256 (Figure 4), the advantage of deeper subgames becomes more pronounced, particularly in the early stages of computation.

Similar patterns emerge for the DCFR algorithm. With 64 iterations (Figure 5), DCFR shows more efficient convergence compared to the Linear update method, but still exhibits clear benefits from increasing subgame depth. With 256 iterations (Figure 6), DCFR achieves lower exploitability across all depth settings, with the full-game solving approach (`max_depth` = ∞) reaching the lowest exploitability.

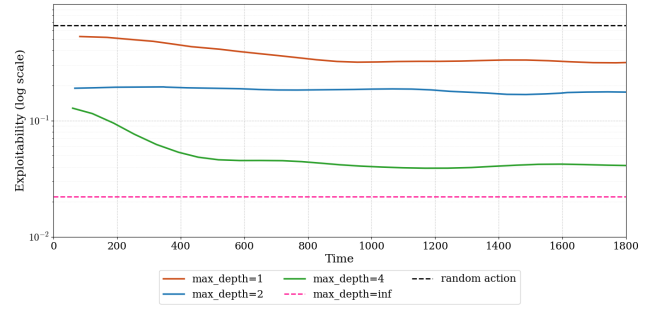


Fig. 3. Exploitability of Linear update method with 64 iterations across different subgame depths (`max_depth` = 1, 2, 4, ∞ , and random) as a function of computational resources.

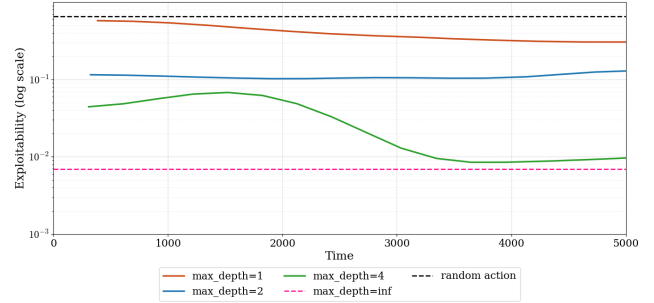


Fig. 4. Exploitability of Linear update method with 256 iterations across different subgame depths (`max_depth` = 1, 2, 4, ∞ , and random) as a function of computational resources.

C. Improvement over Random Baseline

Our results demonstrate that increasing the subgame depth consistently reduces exploitability, though with diminishing returns as depth increases. Compared to the random baseline, the different depths show the following reductions in exploitability (based on DCFR with 256 iterations, which achieved the best results):

- `max_depth` = 1: 61.35% reduction in exploitability
- `max_depth` = 2: 82.73% reduction in exploitability
- `max_depth` = 4: 99.76% reduction in exploitability
- `max_depth` = ∞ : 99.93% reduction in exploitability

D. Computational Efficiency Analysis

While deeper subgames yield better approximations of Nash equilibrium strategies, they also incur significantly higher computational costs. From the figures, we can observe that for a fixed computational budget, a moderate depth of `max_depth` = 2 often provides a good trade-off between solution quality and efficiency.

Specifically, with `max_depth` = 2 using DCFR with 256 iterations, ReBeL achieves 82.73% of the exploitability reduction compared to the random baseline, while requiring substantially less computational resources than the deeper configurations. When we consider `max_depth` = 4, the exploitability reduction reaches 99.76%, approaching the performance of full-game solving (99.93%) but with significantly lower computational demands. This suggests that in many practical applications, limited-depth solving can

TABLE I
EXPLOITABILITY AFTER CONVERGENCE FOR DIFFERENT ALGORITHM CONFIGURATIONS AND SUBGAME DEPTHS IN LIAR'S DICE.

Algorithm	Subgame Depth (<code>max_depth</code>)				
	Random	1	2	4	∞
Linear (64 iter.)	0.655059	0.273370	0.175118	0.040976	0.022257
Linear (256 iter.)	0.655059	0.306475	0.102300	0.011535	0.006942
DCFR (64 iter.)	0.655059	0.279355	0.170436	0.017192	0.012609
DCFR (256 iter.)	0.655059	0.253124	0.113118	0.001591	0.000461

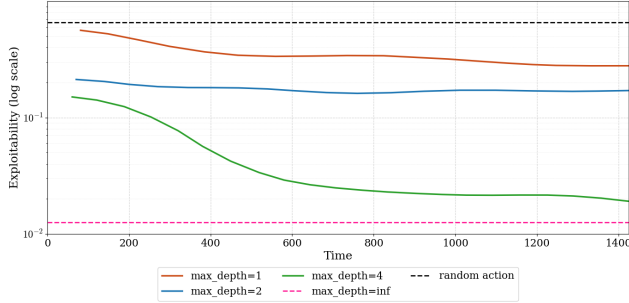


Fig. 5. Exploitability of DCFR algorithm with 64 iterations across different subgame depths (`max_depth` = 1, 2, 4, ∞ , and random) as a function of computational resources.

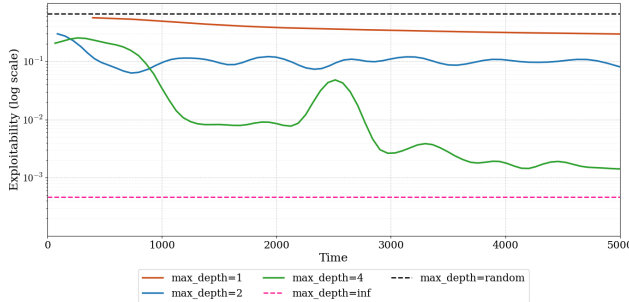


Fig. 6. Exploitability of DCFR algorithm with 256 iterations across different subgame depths (`max_depth` = 1, 2, 4, ∞ , and random) as a function of computational resources.

provide near-optimal results at much lower computational cost.

E. Effect of Iteration Count

Our experiments with different iteration counts (64 vs. 256) show that the benefits of deeper subgames become more pronounced with more iterations. For example, with the DCFR algorithm, increasing iterations from 64 to 256 at `max_depth` = 4 reduces exploitability from 0.017192 to 0.001591, a 90.74% improvement. This indicates that subgame depth and iteration count have a synergistic relationship, where increasing both leads to the best overall performance.

F. Conclusions from Experiments

These experimental results on Liar's Dice confirm our theoretical analysis that subgame depth is a critical hyperparameter in ReBeL. The optimal depth setting depends on the available computational resources and the

desired accuracy. For most practical applications, a moderate depth (`max_depth` = 2 or `max_depth` = 4) provides the best balance between accuracy and efficiency. Specifically, `max_depth` = 4 achieves nearly the same performance as full-game solving (99.76% vs. 99.93% reduction in exploitability) while requiring significantly fewer computational resources.

REFERENCES

- [1] N. Brown, A. Bakhtin, A. Lerer, and Q. Gong, "Combining deep reinforcement learning and search for imperfect-information games," *Advances in neural information processing systems*, vol. 33, pp. 17 057–17 069, 2020.
- [2] G. Tesauro *et al.*, "Temporal difference learning and td-gammon," *Communications of the ACM*, vol. 38, no. 3, pp. 58–68, 1995.
- [3] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot *et al.*, "Mastering the game of go with deep neural networks and tree search," *nature*, vol. 529, no. 7587, pp. 484–489, 2016.
- [4] D. Silver, T. Hubert, J. Schrittwieser, I. Antonoglou, M. Lai, A. Guez, M. Lanctot, L. Sifre, D. Kumaran, T. Graepel *et al.*, "A general reinforcement learning algorithm that masters chess, shogi, and go through self-play," *Science*, vol. 362, no. 6419, pp. 1140–1144, 2018.
- [5] M. Zinkevich, M. Johanson, M. Bowling, and C. Piccione, "Regret minimization in games with incomplete information," *Advances in neural information processing systems*, vol. 20, 2007.
- [6] O. Tammelin, "Solving large imperfect information games using cfr+," *arXiv preprint arXiv:1407.5042*, 2014.
- [7] N. Brown and T. Sandholm, "Solving imperfect-information games via discounted regret minimization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, no. 01, 2019, pp. 1829–1836.
- [8] N. Burch, M. Moravcik, and M. Schmid, "Revisiting cfr+ and alternating updates," *Journal of Artificial Intelligence Research*, vol. 64, pp. 429–443, 2019.
- [9] M. Moravčík, M. Schmid, N. Burch, V. Lisý, D. Morrill, N. Bard, T. Davis, K. Waugh, M. Johanson, and M. Bowling, "Deepstack: Expert-level artificial intelligence in heads-up no-limit poker," *Science*, vol. 356, no. 6337, pp. 508–513, 2017.
- [10] B. Li, Z. Fang, and L. Huang, "RI-cfr: improving action abstraction for imperfect information extensive-form games with reinforcement learning," *arXiv preprint arXiv:2403.04344*, 2024.
- [11] B. Li and L. Huang, "Efficient online pruning and abstraction for imperfect information extensive-form games," in *The Thirteenth International Conference on Learning Representations*.
- [12] J. Perolat, B. De Vylder, D. Hennes, E. Tarassov, F. Strub, V. de Boer, P. Muller, J. T. Connor, N. Burch, T. Anthony *et al.*, "Mastering the game of stratego with model-free multiagent reinforcement learning," *Science*, vol. 378, no. 6623, pp. 990–996, 2022.